The OpenClaw Blueprint v2

Build Your AI Operator — From Zero to Automated Income

by dossdyai

Introduction

AI Agent vs Chatbot — The Real Difference

Most people think AI means ChatGPT. You type a question, it answers. That's a chatbot.

An **AI Agent** is different. It doesn't just answer — it *acts*.

Chatbot	AI Agent
Answers questions	Completes tasks
Forgets everything after session	Has permanent memory
Can't touch your files or apps	Connects to real tools
You do the work after it responds	It does the work for you
One message at a time	Runs on a schedule automatically

ChatGPT is a very smart calculator. OpenClaw is an employee who never sleeps.

Why OpenClaw?

OpenClaw is an open-source AI gateway that connects Claude (by Anthropic) to your real life. It can:

- Read and write your files
- Send emails and messages
- Deploy websites and push code
- Run on a schedule without you touching anything
- Remember everything across all sessions
- Take payments and deliver products automatically

It's not a subscription tool. You own it. You run it. Your data stays yours.

PART 1: SETUP

Chapter 1: What Is OpenClaw?

OpenClaw is a gateway — a bridge between a powerful AI model and the tools you use every day.

When you install OpenClaw and connect it to your Telegram, you get an AI assistant that:

- Lives inside your Telegram (or Discord, or LINE)
- Has access to your computer or VPS
- Can run code, read files, browse the web, send emails
- Works 24/7 — even when your laptop is closed

The core concept: **you give OpenClaw skills, and it uses them when needed.**

Skills are pre-built integrations — Notion, GitHub, Stripe, Google Sheets, and more. Install a skill once, and your AI knows how to use it forever.

Chapter 2: Install on Mac

What You Need

- macOS 12 or newer
- Node.js 18+ (install via nodejs.org)
- A Telegram account

Step 1 — Install OpenClaw

```
npm install -g openclaw
```

Step 2 — Run Setup

```
openclaw setup
```

Follow the prompts:

1. Create a Telegram bot via @BotFather → copy the token
2. Choose Claude Sonnet as your model
3. Set your workspace path (default: `~/.openclaw/workspace`)

Step 3 — Start the Gateway

```
openclaw gateway start
```

Send a message to your Telegram bot. If it responds — you're live.

Step 4 — Auto-start on Login (optional)

```
openclaw gateway install-service
```

Now OpenClaw starts automatically when your Mac boots.

Chapter 3: Install on Windows

What You Need

- Windows 10 or 11
- Node.js 18+ from nodejs.org
- A Telegram account

Step 1 — Open PowerShell as Administrator

Press `Win + X` → choose **Terminal (Admin)** or **PowerShell (Admin)**

Step 2 — Install OpenClaw

```
npm install -g openclaw
```

Step 3 — Run Setup

```
openclaw setup
```

Same steps as Mac — paste your Telegram bot token, choose your model, set workspace path.

Step 4 — Start the Gateway

```
openclaw gateway start
```

Step 5 — Run as Background Service (Windows)

```
# Install pm2 to keep it running
npm install -g pm2
pm2 start "openclaw gateway start" --name openclaw
pm2 save
pm2 startup
```

Copy and run the command it outputs — this makes OpenClaw start on Windows boot.

Troubleshooting Windows

- **"npm not found"** — Restart PowerShell after installing Node.js
- **"Permission denied"** — Run as Administrator
- **Antivirus blocking** — Add `~\AppData\Roaming\npm\` to antivirus exceptions

Chapter 4: Install on VPS — Run 24/7 for $5/Month

A VPS (Virtual Private Server) lets your bot run all the time — even when your laptop is off.

Recommended Providers

- **Hetzner** — €4/month, fast European servers
- **DigitalOcean** — $6/month, easy to use
- **Vultr** — $5/month, flexible

Step 1 — Create a VPS

Choose Ubuntu 22.04. Smallest tier is enough (1 CPU, 1GB RAM).

Step 2 — Connect via SSH

```
ssh root@YOUR_SERVER_IP
```

Step 3 — Install Dependencies

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs git
npm install -g openclaw pm2
```

Step 4 — Run Setup

```
openclaw setup
```

Step 5 — Start as Persistent Service

```
pm2 start "openclaw gateway start" --name openclaw
pm2 save
pm2 startup
# Copy and run the command it outputs
```

Step 6 — Transfer Your Workspace

From your local machine:

```
scp -r ~/.openclaw root@YOUR_SERVER_IP:~/
```

Your bot now runs 24/7. Reboot the VPS — it restarts automatically.

Chapter 5: Telegram — Your Command Center

Telegram is how you control your bot. There are two ways to use it.

Option A: Direct Chat (Simple)

Message your bot directly. Works immediately after setup. Good for personal use.

Option B: Group with Topics (Recommended)

Create a private group, add your bot, enable **Topics mode**. This gives you isolated workspaces:

- 💼 **Business** — products, revenue, marketing tasks
- 🔧 **Dev** — coding, deployment, GitHub work
- 📊 **Research** — monitoring, news, analysis
- 📋 **General** — everything else

Setup steps:

1. Telegram → New Group → add your bot
2. Tap group name → Edit → enable Topics
3. Create one topic per area
4. Via @BotFather: your bot → Group Privacy → turn **OFF**

Each topic maintains separate context. Your business conversations don't mix with dev work.

Sending commands: Just type naturally. Your bot understands plain language:

- *"Summarize my emails from today"*
- *"Create a Notion page for the new product"*
- *"Check GitHub for any failed deployments"*
- *"Run the morning news scout"*

Chapter 6: WebUI on VPS

OpenClaw has a web interface you can access from any browser when running on a VPS.

Accessing the WebUI

After starting OpenClaw on your VPS:

```
openclaw gateway start --port 3000
```

Access it at: `http://YOUR_SERVER_IP:3000`

Secure It with a Reverse Proxy (Recommended)

Never expose port 3000 directly. Use Nginx with a password:

```
apt install -y nginx apache2-utils
htpasswd -c /etc/nginx/.htpasswd admin
```

Create `/etc/nginx/sites-available/openclaw`:

```
server {
    listen 80;
    server_name YOUR_DOMAIN_OR_IP;

    location / {
        auth_basic "OpenClaw";
        auth_basic_user_file /etc/nginx/.htpasswd;
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
    }
}
```

```
ln -s /etc/nginx/sites-available/openclaw /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx
```

What You Can Do in the WebUI

- View conversation history
- Manage Skills (install, update, remove)
- Configure credentials (API keys, tokens)

- Set up and monitor Cron jobs
- View logs and debug issues
- Manage sessions

PART 2: YOUR AI'S BRAIN

Chapter 7: Identity — SOUL.md + USER.md

OpenClaw reads files in your workspace to understand who it is and who it's helping. Two files matter most.

SOUL.md — Your Bot's Personality

```
# SOUL.md

Be genuinely helpful, not performatively helpful.
Skip the "Great question!" — just help.

Have opinions. Disagree when appropriate.
Be resourceful before asking questions.

Earn trust through competence, not filler words.
```

Customize this however you want. The more specific, the better the behavior.

USER.md — Information About You

```
# USER.md

- Name: [Your Name]
- Timezone: Asia/Bangkok
- Preferred language: Thai and English
- Business: [Your business type]
- Communication style: Concise bullet points
- Important tools: Notion, Google Sheets, Stripe
```

The bot reads these files at the start of every session. They persist across reboots — no re-explaining yourself.

IDENTITY.md — Name Your Bot

```
# IDENTITY.md

- Name: [Your bot's name]
- Role: AI operator for [Your Name]
- Emoji: 🤖
```

Chapter 8: Memory System — Your Bot Remembers Everything

By default, AI forgets everything when a session ends. OpenClaw solves this with files.

Layer 1: Daily Notes

`memory/YYYY-MM-DD.md` — automatically created each day. Raw log of what happened.

Layer 2: Long-Term Memory

`MEMORY.md` — your curated, distilled knowledge base. Key decisions, ongoing context, important preferences. Your bot reads this at the start of every session.

Layer 3: Knowledge Graph (PARA System)

A folder structure in `~/life/` :

```
~/life/
  Projects/     ← active, time-bound work (e.g. "new product launch")
  Areas/        ← ongoing responsibilities (e.g. "business finances")
  Resources/    ← reference material (e.g. "product documentation")
  Archives/     ← completed work
```

Create a folder for every person, project, or company that matters. Add a `summary.md` inside each one.

Example:

```
~/life/
  Projects/
    openclaw-blueprint/
      summary.md    ← what this project is, decisions made, current status
  Areas/
    business-finances/
      summary.md
```

Nightly Memory Update (Automatic)

Set a cron job for 2AM:

> *"Review today's conversations. Update ~/life/ with new decisions and context. Update MEMORY.md with any significant learnings."*

Your bot builds a richer picture of your world over time — getting more useful the longer you run it.

Chapter 9: Behavior Rules — AGENTS.md

`AGENTS.md` defines how your bot operates. Key rules to set:

Plan Before Acting

For multi-step tasks, write a plan to `tasks/todo.md` first. Verify before starting. Prevents the bot from going in the wrong direction on complex work.

Self-Improvement Loop

After every correction, update `tasks/lessons.md` with what was learned. Over time, your bot stops making the same mistakes.

Verify Before Done

Never mark a task complete without proving it works. Run checks, review output.

Autonomous Problem Solving

For bugs and technical problems: fix them. Don't ask for hand-holding. Point at errors and logs — then resolve.

Example AGENTS.md entry:

```
## Rules
1. Plan first → tasks/todo.md → confirm before starting
2. After corrections → update tasks/lessons.md
3. Never mark done without testing
4. For any external action (email, tweet, deploy) → confirm first
```

PART 3: AUTOMATION

Chapter 10: Cron Jobs + Heartbeat

Your bot can run tasks on a schedule — no input required from you.

Setting Up a Cron Job

```
openclaw cron add \
  --name "morning-scout" \
  --cron "0 8 * * *" \
  --tz "Asia/Bangkok" \
  --message "Fetch top AI/tech news from HackerNews and Reddit. Summarize top 5
stories. Send to Telegram."
```

Useful Cron Jobs to Set Up

Name	Schedule	Task
morning-scout	8AM daily	News digest
memory-update	2AM daily	Update ~/life/ + MEMORY.md
weekly-review	9AM Sunday	Week summary + project status
price-monitor	9AM daily	Track competitor prices or stocks

Heartbeat System

Edit `HEARTBEAT.md` — your bot checks this file regularly and acts on whatever is listed:

```
# HEARTBEAT.md
- Check inbox for urgent emails
- Review calendar for events in next 24h
- Flag any overdue tasks in tasks/todo.md
```

OpenClaw polls the heartbeat prompt periodically. If there's nothing, it stays quiet. If something needs attention, it messages you.

Chapter 11: Morning Scout Setup

A practical example — setting up a daily news digest that arrives every morning.

The Setup

```
openclaw cron add \
  --name "morning-scout" \
  --cron "0 8 * * *" \
  --tz "Asia/Bangkok" \
  --message "Search HackerNews top stories and Reddit r/artificial for posts from the last 24h. Filter for AI tools, automation, and business opportunities. Write a clean 5-point summary with links. Send to Telegram."
```

Customize Your Scout

Change the topics to match your business:

- E-commerce: monitor competitor product launches
- Finance: track crypto/stock news
- Marketing: TikTok trends, viral content patterns
- Tech: new tool releases, GitHub trending

The scout runs while you sleep. You wake up already informed.

PART 4: INTEGRATIONS

Chapter 12: Notion

Notion becomes your bot's organized second brain — a place to store projects, databases, and structured notes.

Setup

1. notion.so/my-integrations → Create integration → copy API key
2. `openclaw config set NOTION_API_KEY your_key_here`
3. Share any Notion page with your integration to grant access

What Your Bot Can Do

- Create and update pages automatically
- Query and filter databases
- Add checklist items, track progress
- Write reports and summaries directly to Notion

Recommended Workspace Structure

- **Hub page** — links to all projects
- **Project pages** — one per active project
- **Databases** — tasks, ideas, content calendar, CRM

Chapter 13: Google Sheets, Drive + Docs

Connect your bot to Google's suite using a service account.

Setup

1. Go to console.cloud.google.com → Create a project
2. Enable: Google Sheets API, Google Drive API, Google Docs API
3. Create a Service Account → download JSON key file
4. Share any Sheet/Drive folder/Doc with the service account email

Store the JSON key at:

```
~/Library/Application Support/gspread/service_account.json
```

Google Sheets

Your bot can read and write any spreadsheet shared with it:

- Log sales data automatically
- Read inventory and update stock levels
- Create weekly reports from raw data
- Track KPIs without manual input

Google Drive

- Upload files automatically (reports, exports, PDFs)
- Organize and move files on command
- Share files with specific people

Google Docs

- Generate reports and write documents automatically
- Update existing documents with new data
- Create templated contracts or briefs on demand

Chapter 14: Gmail

Send and read emails automatically using OAuth2.

Setup

1. console.cloud.google.com → Enable Gmail API
2. Create OAuth2 credentials → download `credentials.json`

3. Configure via Himalaya (OpenClaw's email skill):

```
# The himalaya skill handles Gmail OAuth2
openclaw skill install himalaya
```

What Your Bot Can Do

- Send emails on your behalf
- Read and summarize your inbox
- Flag urgent messages and notify you on Telegram
- Draft and send follow-ups automatically
- Search for specific emails by topic or sender

Useful Automations

- **Daily inbox summary** at 9AM — top 5 emails that need attention
- **Auto-response templates** for common inquiries
- **Order confirmation emails** triggered by Stripe payments

Chapter 15: Image Generation

Your bot can generate images using AI — for products, social media, thumbnails, and marketing.

Gemini Image Generation

```
openclaw config set GEMINI_API_KEY your_key
```

Then in Telegram:

> *"Generate a product banner for the OpenClaw Blueprint guide. Dark background, purple accent, clean modern design."*

DALL-E (via OpenAI)

```
openclaw config set OPENAI_API_KEY your_key
```

Use Cases

- Product mockups and thumbnails
- Social media visuals
- Blog post illustrations
- Presentation slides
- TikTok video covers

Prompt Tips for Best Results

- Be specific about style: *"minimalist", "dark mode", "photorealistic"*
- Include dimensions: *"1080x1080 for Instagram"*
- Reference your brand: *"dossdyai brand — dark background, purple accents"*

Chapter 16: GitHub + Vercel — Your Bot Deploys Code

Give your bot its own GitHub and Vercel accounts so it can push code and deploy apps — separate from your personal accounts.

Why Separate Accounts?

If something goes wrong, revoke the bot's tokens. Your personal accounts stay untouched.

Setup

Bot's GitHub account:

- Create with a dedicated email (e.g. yourbot@gmail.com)
- Username: yourname-bot
- Settings → Developer Settings → Tokens (classic)
- Scopes: `repo`, `workflow`, `read:org`

Bot's Vercel account:

- Sign up at vercel.com with the bot's GitHub
- Account → Settings → Tokens → Create

Store both in `~/.openclaw/secrets/accounts.md` (chmod 600).

What Your Bot Can Do

- Create GitHub repositories
- Push code and commit changes
- Deploy to Vercel — get a live URL in 60 seconds
- Create GitHub Pages sites
- Manage pull requests

Practical Example

You say: *"Build a landing page for my new product and deploy it"*

Your bot:

1. Creates HTML file in workspace
2. Creates GitHub repo
3. Pushes code
4. Deploys to Vercel
5. Gives you the live URL

All in under 2 minutes.

Chapter 17: n8n — Workflow Automation

n8n is a workflow automation tool (like Zapier but self-hosted and free). Combined with OpenClaw, you get AI-powered automated workflows.

Install n8n on VPS

```
npm install -g n8n
pm2 start n8n --name n8n
```

Access at: `http://YOUR_SERVER_IP:5678`

Connect OpenClaw to n8n

Your bot can trigger n8n workflows via webhook:

```
# In n8n: create a workflow with a Webhook trigger
# Copy the webhook URL
# In OpenClaw, tell the bot:
# "Trigger the CRM update workflow" → bot calls the n8n webhook
```

Practical Workflows to Build

New customer workflow: Stripe payment → n8n → Add to Google Sheets CRM → Send welcome email → Create Notion page

Content workflow: Bot generates content idea → n8n → Post to Notion draft → Notify Telegram → Schedule post

Sales report workflow: Daily at 9AM → n8n → Pull Google Sheets data → Format report → Send to Telegram

The Power Combination

OpenClaw handles the AI thinking. n8n handles the multi-step execution. Together you get:

> *"Analyze my sales this week and send a report to my business partner"*

Bot processes the data → triggers n8n → n8n pulls Sheets data → formats → sends email → done.

PART 5: MAKE MONEY

Chapter 18: Stripe — Accept Payments

Step 1 — Create a Stripe Account

- stripe.com → Create account
- Complete onboarding (requires a website URL)
- Enable 2FA

Step 2 — Get API Keys

Dashboard → Developers → API Keys

- **Publishable key** (`pk_`) — safe to use in frontend code
- **Secret key** (`sk_`) — never expose this. Store securely.

```
# Store safely — never commit to Git
```

```
chmod 600 ~/.openclaw/secrets/accounts.md
```

Step 3 — Create a Product

Dashboard → Product Catalog → Add Product

- Name, description, price
- Type: One-time payment
- Copy the Price ID (`price_...`)

Step 4 — Create a Payment Link

Dashboard → Payment Links → Create

- Select your product
- Set redirect URL to your thank-you page
- Copy the payment link

Test Before Going Live

Use Stripe test mode and card: `4242 4242 4242 4242` Any future expiry date, any CVC.

Run a full test purchase before switching to live mode.

Chapter 19: Auto-Delivery Webhook

When someone buys, they should receive their product instantly — automatically, with no action from you.

How It Works

```
Payment succeeds
      ↓
Stripe fires: checkout.session.completed
      ↓
Your webhook endpoint receives it
      ↓
Sends email to buyer with product download/access
      ↓
Buyer receives product
      ↓
You receive nothing — you do nothing
```

Deploy a Webhook (Vercel Serverless)

Create `api/webhook.js` :

```
export default async function handler(req, res) {
  const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
  const event = stripe.webhooks.constructEvent(
    await getRawBody(req),
    req.headers['stripe-signature'],
    process.env.STRIPE_WEBHOOK_SECRET
  );
```

```
  if (event.type === 'checkout.session.completed') {
    const { email, name } = event.data.object.customer_details;
    await sendDeliveryEmail(email, name); // via Resend API
  }

  res.json({ received: true });
}
```

Email Service: Resend (Free Tier)

- Sign up at resend.com (3,000 emails/month free)
- Create API key
- Add to Vercel environment variables

Register Webhook in Stripe

Dashboard → Developers → Webhooks → Add endpoint

- URL: your Vercel deployment URL + `/api/webhook`
- Events: `checkout.session.completed`
- Copy the webhook signing secret (`whsec_...`)

Chapter 20: Build Your Landing Page

A landing page is the one page someone sees before clicking "Buy." It needs to do one thing: convert interest into payment.

The Essential Structure

```
HEADLINE          ← What it is + who it's for (10 words max)
SUBHEADLINE       ← The outcome they'll get
SOCIAL PROOF      ← Real results or credentials
WHO IT'S FOR      ← 4–5 bullet points of ideal customers
WHAT'S INSIDE     ← Table or list of contents
BUY BUTTON        ← Price + one CTA
FAQ               ← Kill the last objections
```

Deployment

1. Write HTML file locally
2. Push to GitHub (bot's account)
3. Enable GitHub Pages — free hosting instantly
4. Or deploy to Vercel — same result, faster CDN

```
# Bot can deploy your landing page with one message:
# "Deploy the landing page to GitHub Pages"
```

What Makes People Buy

- **Results first** — show the outcome before explaining what's inside
- **One CTA** — one button. Don't give multiple options.

- **Price anchoring** — show original price crossed out, sale price highlighted
- **Specificity** — "23 chapters" beats "comprehensive guide"

Chapter 21: Your First Product — Full Launch Checklist

Week 1 — Build

- [ ] Decide what to sell (guide, course, tool, template)
- [ ] Create the product (write it, build it — your bot helps)
- [ ] Create Stripe product + set price
- [ ] Write landing page copy
- [ ] Deploy landing page

Week 1 — Wire Up

- [ ] Deploy webhook (Vercel)
- [ ] Set up email delivery (Resend)
- [ ] Test purchase with card `4242 4242 4242 4242`
- [ ] Confirm delivery email arrives
- [ ] Check thank-you page + download works

Week 2 — Launch

- [ ] Switch Stripe to live mode
- [ ] Record video (show result, not the how)
- [ ] Post on TikTok / Instagram / X
- [ ] Pin purchase link in comments
- [ ] Reply to every comment in first 30 minutes

Ongoing

- [ ] Bot monitors Stripe dashboard daily
- [ ] Weekly revenue report via cron
- [ ] Bot handles FAQ replies in DMs

The entire system runs itself after this. Your only job is to drive traffic.

PART 6: MAINTENANCE

Chapter 22: Cost Optimization

Running OpenClaw costs money mainly through AI model API calls. Here's how to keep costs low without sacrificing quality.

Choose the Right Model for Each Task

Task	Recommended Model	Cost Level
Complex analysis, writing	Claude Sonnet	Medium

Simple tasks, quick replies	Claude Haiku	Low
Code generation	Claude Sonnet	Medium
Cron jobs, monitoring	Claude Haiku	Low
Image generation	Gemini Flash	Very Low

Rule: Use the smallest model that does the job well.

Reduce Token Usage

- Keep MEMORY.md concise — trim old, irrelevant entries
- Don't load all memory files on every request
- Use cron jobs during off-peak hours (cheaper or same cost, less latency)
- Batch similar tasks into one message instead of multiple

Monitor Your Spend

- Anthropic console → Usage dashboard
- Set spending limits: console.anthropic.com → Settings → Limits
- Review weekly — spike in cost usually means a runaway cron job

Free + Low-Cost Options

- **Gemini Flash** — very cheap, good for image generation and simple tasks
- **Groq** — extremely fast, free tier available, good for quick tasks
- **Local models via Ollama** — run models locally on your Mac/VPS, zero API cost

To use a local model:

```
# Install Ollama
curl -fsSL https://ollama.ai/install.sh | sh
ollama pull llama3.2

# Configure OpenClaw to use local model
openclaw config set DEFAULT_MODEL ollama/llama3.2
```

Chapter 23: Troubleshooting Guide

Gateway Issues

Problem: Bot not responding in Telegram

```
# Check if gateway is running
openclaw gateway status

# Restart if needed
openclaw gateway restart

# View logs
openclaw gateway logs
```

Problem: Gateway crashes on startup

```
# Check for port conflicts
lsof -i :3000

# Try a different port
openclaw gateway start --port 3001
```

Pairing Issues

Problem: "Gateway not paired" error

```
# Re-run setup
openclaw setup

# Or manually re-pair
openclaw gateway pair
```

Problem: Telegram bot not receiving messages

- Check @BotFather → your bot → Group Privacy → must be **OFF**
- Confirm bot was added to the group
- Try sending a message directly to the bot (not the group)

VPS Issues

Problem: PM2 not persisting after reboot

```
pm2 save
pm2 startup
# Run the command it outputs
```

Problem: OpenClaw running but not reachable

```
# Check if process is running
pm2 status

# Check port is open
ufw allow 3000
ufw status
```

Token + API Issues

Problem: "Invalid API key" errors

- Check key is correct: no spaces, no trailing newlines
- Make sure you're using the right key (test vs live for Stripe)
- Re-enter via `openclaw config set KEY_NAME value`

Problem: High API costs suddenly

- Check cron jobs — one may be looping
- `openclaw cron list` → look for jobs running too frequently
- Check MEMORY.md — if very large, trim it

Fresh Install on VPS

If something is broken and you want to start fresh:

```
# Stop everything
pm2 delete all

# Remove OpenClaw
npm uninstall -g openclaw

# Clear workspace (WARNING: backs up first)
cp -r ~/.openclaw ~/.openclaw-backup-$(date +%Y%m%d)
rm -rf ~/.openclaw

# Reinstall
npm install -g openclaw
openclaw setup
```

Final Words

You now have everything to run an AI-powered operation that works while you sleep.

The path forward:

1. Set up OpenClaw (Mac, Windows, or VPS)
2. Add your identity files — SOUL.md, USER.md
3. Connect one integration (start with Notion or Google Sheets)
4. Set up one cron job (morning scout is the easiest)
5. Build one product and wire up Stripe
6. Post one video showing the result

That's it. Everything else is iteration.

The bot gets smarter as you use it. The system gets more automated as you build it. The income grows as you ship products.

Start simple. Prove it works. Then scale.

The OpenClaw Blueprint v2 — by dossdyai Version 2.0 — 2026 English + Thai editions